

OOP Darbas

1 Paleidimo instrukcija	1
1.1 V0.1 užduotis {	1
1.2 }	1
1.3 V0.2 užduotis {	1
1.4 }	2
1.5 V0.3 užduotis {	2
1.6 }	2
1.7 V.04 užduotis {	2
1.8 Tyrimas 1	2
1.9 Tyrimas 2	2
1.10 }	2
1.11 V1.0 užduotis {	2
1.12 }	2
1.13 V1.1 užduotis {	2
1.14 }	3
1.15 V1.2 užduotis {	3
1.16 }	4
1.17 V1.5 užduotis {	4
1.18 }	5
2 Hierarchical Index	7
2.1 Class Hierarchy	7
3 Class Index	9
3.1 Class List	9
4 File Index	11
4.1 File List	11
5 Class Documentation	13
5.1 duomenys_m Struct Reference	13
5.1.1 Detailed Description	13
5.1.2 Member Data Documentation	13
5.1.2.1 egz	13
5.1.2.2 nd	13
5.1.2.3 nd_sk	13
5.1.2.4 pavarde	13
5.1.2.5 vardas	13
5.2 Studentai Class Reference	14
5.3 Zmogus Class Reference	14
5.3.1 Detailed Description	14
5.3.2 Constructor & Destructor Documentation	14
5.3.2.1 ~Zmogus()	14
5.3.3 Member Function Documentation	14

5.3.3.1 getPavarde()	14
5.3.3.2 getVardas()	14
5.3.3.3 setPavarde()	14
5.3.3.4 setVardas()	14
5.3.4 Member Data Documentation	15
5.3.4.1 pavarde	15
5.3.4.2 vardas	15
6 File Documentation	17
6.1 duomenys.cpp File Reference	17
6.1.1 Function Documentation	18
6.1.1.1 dalina()	18
6.1.1.2 galutinis_med() [1/2]	18
6.1.1.3 galutinis_med() [2/2]	18
6.1.1.4 galutinis_vid() [1/2]	18
6.1.1.5 galutinis_vid() [2/2]	18
6.1.1.6 generuoti_paz()	18
6.1.1.7 generuoti_paz_ranka()	18
6.1.1.8 generuoti_vard()	19
6.1.1.9 make_file()	19
6.1.1.10 meniu()	19
6.1.1.11 operator<<()	19
6.1.1.12 operator==()	19
6.1.1.13 operator>>()	19
6.1.1.14 read()	19
6.1.1.15 readas()	19
6.1.1.16 sortas()	20
6.1.1.17 sortass()	20
6.1.1.18 split_into_two_containers()	20
6.1.1.19 ss_write()	20
6.1.1.20 testavimas()	20
6.1.1.21 write()	20
6.1.1.22 write_file()	20
6.1.1.23 write_to_file()	21
6.1.2 Variable Documentation	21
6.1.2.1 umap	21
6.2 duomenys.cpp	21
6.3 duomenys.h File Reference	31
6.3.1 Function Documentation	32
6.3.1.1 galutinis_med()	32
6.3.1.2 galutinis_vid()	32
6.3.1.3 generuoti_paz()	32

6.3.1.4 generuoti_paz_ranka()	32
6.3.1.5 generuoti_vard()	32
6.3.1.6 make_file()	32
6.3.1.7 meniu()	32
6.3.1.8 read()	33
6.3.1.9 sortas()	33
6.3.1.10 ss_write()	33
6.3.1.11 write()	33
6.3.1.12 write_file()	33
6.3.2 Variable Documentation	33
6.3.2.1 test_file_location	33
6.4 duomenys.h	33
6.5 pirma_uzd.cpp File Reference	35
6.5.1 Function Documentation	35
6.5.1.1 main()	35
6.5.2 Variable Documentation	35
6.5.2.1 studentai	35
6.6 pirma_uzd.cpp	35
6.7 README.md File Reference	35
6.8 saugojimas_masyve.cpp File Reference	35
6.8.1 Function Documentation	36
6.8.1.1 main()	36
6.9 saugojimas_masyve.cpp	36
6.10 saugojimas_masyve.h File Reference	36
6.10.1 Function Documentation	36
6.10.1.1 galutinis_med1()	36
6.10.1.2 galutinis_vid1()	37
6.10.1.3 ivedimas()	37
6.10.1.4 write1()	37
6.10.2 Variable Documentation	37
6.10.2.1 B	37
6.10.2.2 zmn_sk	37
6.11 saugojimas_masyve.h	37
6.12 zmogus.h File Reference	38
6.13 zmogus.h	38

Chapter 1

Paleidimo instrukcija

1. Atsisiųskite programos versiją iš Releases.

2. Išsarchyvuokite failus.

3. Atsidarykite aplanką.

4.1 Paleiskite programą, paspaudus "run.bat".

4.2.1 Norint pasileisti programą per terminalą - užkomentuokite pirmą "duomenys.h" eilutę ir atkomentuokite antrą "duomenys.cpp".

4.2.2 Norint sukompiliuoti programą per terminalą, įveskite komandą `g++ -O3 -o programa pirma_uzd.cpp`.

4.2.3 Norint paleisti programą, per terminalą įveskite komandą `./programa`.

4.3 Norint pasileisti programą per jūsų pasirinktą editorių - užkomentuokite antrą "duomenys.cpp" eilutę ir atkomentuokite pirmą "duomenys.h".

1.1 V0.1 užduotis {

V0.1 užduotyje buvo parašytas programos pagrindas, kuris sugeneruoja duomenis ir nuskaitytuos juos iš failo ir išveda juos į konsolę.

1.2 }

1.3 V0.2 užduotis {

V0.2 užduotyje buvo pridėtas nuskaitymas iš jau prieš tai nugeneruoto failo ir išvedimas į konsolę.

1.4 }

1.5 V0.3 užduotis {

V0.3 užduotyje buvo pridėti try catch blokai, kurie užtikrino programos veikimą. Taip pat programa buvo surūšiuota į header ir cpp failus.

1.6 }

1.7 V.04 užduotis {

1.8 Tyrimas 1

Failų (1000, 10000, 100000, 1000000, 10000000) generavimas vidutiniškai (imant trijų generavimų vidurkį) užtruko 46.6222 s.

1.9 Tyrimas 2

visi testai buvo atliekami po 3 kartus ir paimtas jų vidurkis.

1.10 }

1.11 V1.0 užduotis {

CPU: Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz 2.30 GHz

RAM: 16.0 GB

SSD: KXG6AZNV512G TOSHIBA M.2 2280

1.12 }

1.13 V1.1 užduotis {

Tyrimai atlikti su O1, O2, O3 optimizacijomis. O1 flag'o exe failo dydis - 242KB

O2 flag'o exe failo dydis - 206KB

O3 flag'o exe failo dydis - 204KB

Palyginus rezultatus su struktūros ir klasės optimizacijomis, pastebėta, kad klasės optimizacija yra greitesnė (mažiau 2 kartus) už struktūros optimizaciją. Taip pat pastebėta, kad O3 optimizacija yra greitesnė už O2 ir O1 optimizacijas.

1.14 }

1.15 V1.2 užduotis {

Copy konstruktorius - is vieno objekto nukopijuoja i kita objekta.

Copy assignment operatorius - is vieno objekto nukopijuoja i kita objekta, kai objektai jau egzistuoja.

Move konstruktorius - is vieno objekto perkelia i kita objekta, kai objektas neegzistuoja, ir objektas, is kurio keliame, yra sunaikinamas.

Move assignment operatorius - is vieno objekto perkelia i kita objekta, kai objektas jau egzistuoja, ir objektas, is kurio keliame, yra sunaikinamas.

Destruktorius - sunaikina objekta, kai jis nebereikalingas.

Input operatorius - leidzia nuskaityti objekta is failo ir perdengia default input operatoriu.

Output operatorius - leidzia isvesti objekta i faila ir perdengia default output operatoriu.

1000

laikas	sek.
Nuskaitymo laikas:	0.0559618s
rusiavimo didejimo tvarka laikas:	0.0007691s
dalinimo laikas:	0.0018016s
Is viso laiko:	0.0585325s

10000

laikas	sek.
Nuskaitymo laikas:	0.0935924s
rusiavimo didejimo tvarka laikas:	0.09089s
dalinimo laikas:	0.0085245s
Is viso laiko:	0.109313s

100000

laikas	sek.
Nuskaitymo laikas:	0.491717s
rusiavimo didejimo tvarka laikas:	0.09089s
dalinimo laikas:	0.0994205s
Is viso laiko:	0.682028s

1000000

laikas	sek.
Nuskaitymo laikas:	3.03551s
rusiavimo didejimo tvarka laikas:	0.907027s
dalinimo laikas:	0.963172s
Is viso laiko:	4.90571s

10000000

laikas	sek.
Nuskaitymo laikas:	31.1019s
rusiavimo didejimo tvarka laikas:	9.76324s
dalinimo laikas:	10.3145s
Is viso laiko:	51.1796s

Palyginus su praeita versija, pastebime, kad programa veikia greičiau.

Isvada - naudojant "rule of five" bei perdengiant ivedimo ir isvedimo operatorius, programa veikia greičiau.

1.16 }

1.17 V1.5 užduotis {

Sukurta Abstrakti klase "Zmogus", kuri aprasyta "zmogus.h" faile. Is sios klases isejo derived klase "Studentas".
1000

laikas	sek.
Nuskaitymo laikas:	0.0384418s
rusiavimo didejimo tvarka laikas:	0.0011993s
dalinimo laikas:	0.0020543s
Is viso laiko:	0.0416954s

10000

laikas	sek.
Nuskaitymo laikas:	0.0762262s
rusiavimo didejimo tvarka laikas:	0.0086051s
dalinimo laikas:	0.010063s
Is viso laiko:	0.010063s

100000

laikas	sek.
Nuskaitymo laikas:	0.261733s
rusiavimo didejimo tvarka laikas:	0.10739s
dalinimo laikas:	0.114907s
Is viso laiko:	0.484031s

1000000

laikas	sek.
Nuskaitymo laikas:	2.03144s
rusiavimo didejimo tvarka laikas:	1.28536s
dalinimo laikas:	1.3503s
Is viso laiko:	4.6671s

10000000

laikas	sek.
Nuskaitymo laikas:	20.8932s

laikas			sek.
rusiavimo laikas:	didejimo	tvarka	13.888s
dalinimo laikas:			14.558s
Is viso laiko:			49.3392s

Atlikus tyrimus pastebima, kad programa veikia greičiau nei praeita versija. taip pat prideta nauja abstrakti klase "Zmogus".

1.18 }

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

duomenys_m	13
Zmogus	14
Studentai	14

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

duomenys_m	13
Studentai	
Klases	14
Zmogus	14

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

duomenys.cpp	17
duomenys.h	31
pirma_uzd.cpp	35
saugojimas_masyve.cpp	35
saugojimas_masyve.h	36
zmogus.h	38

Chapter 5

Class Documentation

5.1 duomenys_m Struct Reference

```
#include <saugojimas_masyve.h>
```

Public Attributes

- std::string [vardas](#)
- std::string [pavarde](#)
- int [nd](#) [100]
- int [nd_sk](#)
- int [egz](#)

5.1.1 Detailed Description

Definition at line [14](#) of file [saugojimas_masyve.h](#).

5.1.2 Member Data Documentation

5.1.2.1 egz

```
int duomenys_m::egz
```

Definition at line [18](#) of file [saugojimas_masyve.h](#).

5.1.2.2 nd

```
int duomenys_m::nd[100]
```

Definition at line [16](#) of file [saugojimas_masyve.h](#).

5.1.2.3 nd_sk

```
int duomenys_m::nd_sk
```

Definition at line [17](#) of file [saugojimas_masyve.h](#).

5.1.2.4 pavarde

```
std::string duomenys_m::pavarde
```

Definition at line [15](#) of file [saugojimas_masyve.h](#).

5.1.2.5 vardas

```
std::string duomenys_m::vardas
```

Definition at line [15](#) of file [saugojimas_masyve.h](#).

The documentation for this struct was generated from the following file:

- [saugojimas_masyve.h](#)

5.2 Studentai Class Reference

klases.

```
#include <duomenys.h>
```

Inheritance diagram for Studentai:

5.3 Zmogus Class Reference

```
#include <zmogus.h>
```

Inheritance diagram for Zmogus:

Public Member Functions

- virtual string [getVardas](#) () const =0
- virtual string [getPavarde](#) () const =0
- virtual void [setVardas](#) (const string &v)=0
- virtual void [setPavarde](#) (const string &p)=0
- virtual [~Zmogus](#) ()=default

Protected Attributes

- string [vardas](#)
- string [pavarde](#)

5.3.1 Detailed Description

Definition at line 7 of file [zmogus.h](#).

5.3.2 Constructor & Destructor Documentation

5.3.2.1 ~Zmogus()

```
virtual Zmogus::~Zmogus () [virtual], [default]
```

5.3.3 Member Function Documentation

5.3.3.1 getPavarde()

```
virtual string Zmogus::getPavarde () const [pure virtual]
```

Implemented in [Studentai](#).

5.3.3.2 getVardas()

```
virtual string Zmogus::getVardas () const [pure virtual]
```

Implemented in [Studentai](#).

5.3.3.3 setPavarde()

```
virtual void Zmogus::setPavarde (  
    const string & p) [pure virtual]
```

Implemented in [Studentai](#).

5.3.3.4 setVardas()

```
virtual void Zmogus::setVardas (  
    const string & v) [pure virtual]
```

Implemented in [Studentai](#).

5.3.4 Member Data Documentation

5.3.4.1 pavarde

`string Zmogus::pavarde` [protected]

Definition at line 10 of file [zmogus.h](#).

5.3.4.2 vardas

`string Zmogus::vardas` [protected]

Definition at line 9 of file [zmogus.h](#).

The documentation for this class was generated from the following file:

- [zmogus.h](#)

Chapter 6

File Documentation

6.1 duomenys.cpp File Reference

```
#include "duomenys.h"
#include "zmogus.h"
#include "saugojimas_masyve.h"
Include dependency graph for duomenys.cpp:
```

Functions

- ostream & [operator<<](#) (ostream &os, const [Studentai](#) &student)
- std::ifstream & [operator>>](#) (std::ifstream &in, [Studentai](#) &student)
- bool [operator==](#) (const [Studentai](#) &a, const [Studentai](#) &b)
- template<typename Container>
void [readas](#) (const string &filename, Container &[studentai](#))
- template<typename Container>
double [dalina](#) (Container &[studentai](#), Container &blogis)
- template<typename Container>
void [sortass](#) (Container &[studentai](#))
- template<typename Container>
void [write_to_file](#) (const string &filename, const Container &[studentai](#))
- template<typename Container>
void [split_into_two_containers](#) (Container &[studentai](#), Container &vargsiukai, Container &kietiakai)
- void [testavimas](#) (vector< [Studentai](#) > &[studentai](#))
- void [menui](#) (vector< [Studentai](#) > &[studentai](#))
- void [galutinis_vid](#) (vector< int > nd, int egz, vector< [Studentai](#) > &[studentai](#))
- double [galutinis_vid](#) (vector< int > nd, int egz)
- void [galutinis_med](#) (vector< int > nd, int egz, vector< [Studentai](#) > &[studentai](#))
- double [galutinis_med](#) (vector< int > nd, int egz)
- void [read](#) (const string &filename, vector< [Studentai](#) > &[studentai](#))
- void [ss_write](#) (const string &filename, vector< [Studentai](#) > &[studentai](#))
- void [write](#) (vector< [Studentai](#) > &[studentai](#))
- void [write_file](#) (const string &filename, vector< [Studentai](#) > &[studentai](#), int nd_skaicius)
- void [make_file](#) (vector< [Studentai](#) > &[studentai](#), const string &filename, int mok_sk, int paz_sk)
- void [generuoti_paz](#) ([Studentai](#) &student, int paz_sk)
- void [generuoti_paz_ranka](#) (vector< [Studentai](#) > &[studentai](#))
- void [generuoti_vard](#) (vector< [Studentai](#) > &[studentai](#), int paz_sk1, int mok_sk)
- void [sortas](#) (vector< [Studentai](#) > &[studentai](#))

Variables

- std::unordered_map< int, string > [umap](#)

6.1.1 Function Documentation

6.1.1.1 dalina()

```
template<typename Container>
double dalina (
    Container & studentai,
    Container & blogis)
```

Definition at line 196 of file [duomenys.cpp](#).

Here is the caller graph for this function:

6.1.1.2 galutinis_med() [1/2]

```
double galutinis_med (
    vector< int > nd,
    int egz)
```

Definition at line 636 of file [duomenys.cpp](#).

6.1.1.3 galutinis_med() [2/2]

```
void galutinis_med (
    vector< int > nd,
    int egz,
    vector< Studentai > & studentai)
```

Definition at line 620 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.1.1.4 galutinis_vid() [1/2]

```
double galutinis_vid (
    vector< int > nd,
    int egz)
```

Definition at line 606 of file [duomenys.cpp](#).

6.1.1.5 galutinis_vid() [2/2]

```
void galutinis_vid (
    vector< int > nd,
    int egz,
    vector< Studentai > & studentai)
```

Definition at line 589 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.1.1.6 generuoti_paz()

```
void generuoti_paz (
    Studentai & student,
    int paz_sk)
```

Definition at line 744 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.1.1.7 generuoti_paz_ranka()

```
void generuoti_paz_ranka (
    vector< Studentai > & studentai)
```

Definition at line 760 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.1.1.8 generuoti_var()

```
void generuoti_var (
    vector< Studentai > & studentai,
    int paz_sk1,
    int mok_sk)
```

Definition at line 788 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.1.1.9 make_file()

```
void make_file (
    vector< Studentai > & studentai,
    const string & filename,
    int mok_sk,
    int paz_sk)
```

Definition at line 739 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.1.1.10 meniu()

```
void meniu (
    vector< Studentai > & studentai)
```

Definition at line 337 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.1.1.11 operator<<()

```
ostream & operator<< (
    ostream & os,
    const Studentai & student)
```

Definition at line 103 of file [duomenys.cpp](#).

Here is the call graph for this function:

6.1.1.12 operator==()

```
bool operator== (
    const Studentai & a,
    const Studentai & b)
```

Definition at line 153 of file [duomenys.cpp](#).

6.1.1.13 operator>>()

```
std::ifstream & operator>> (
    std::ifstream & in,
    Studentai & student)
```

Definition at line 112 of file [duomenys.cpp](#).

6.1.1.14 read()

```
void read (
    const string & filename,
    vector< Studentai > & studentai)
```

Definition at line 650 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.1.1.15 readas()

```
template<typename Container>
void readas (
```

```
const string & filename,
Container & studentai)
```

Definition at line 167 of file [duomenys.cpp](#).

Here is the caller graph for this function:

6.1.1.16 sortas()

```
void sortas (
    vector< Studentai > & studentai)
```

Definition at line 820 of file [duomenys.cpp](#).

Here is the caller graph for this function:

6.1.1.17 sortass()

```
template<typename Container>
void sortass (
    Container & studentai)
```

Definition at line 213 of file [duomenys.cpp](#).

Here is the caller graph for this function:

6.1.1.18 split_into_two_containers()

```
template<typename Container>
void split_into_two_containers (
    Container & studentai,
    Container & vargsiukai,
    Container & kietiakai)
```

Definition at line 267 of file [duomenys.cpp](#).

Here is the caller graph for this function:

6.1.1.19 ss_write()

```
void ss_write (
    const string & filename,
    vector< Studentai > & studentai)
```

Definition at line 682 of file [duomenys.cpp](#).

Here is the caller graph for this function:

6.1.1.20 testavimas()

```
void testavimas (
    vector< Studentai > & studentai)
```

Definition at line 285 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.1.1.21 write()

```
void write (
    vector< Studentai > & studentai)
```

Definition at line 702 of file [duomenys.cpp](#).

Here is the caller graph for this function:

6.1.1.22 write_file()

```
void write_file (
    const string & filename,
    vector< Studentai > & studentai,
    int nd_skaicius)
```

Definition at line 711 of file [duomenys.cpp](#).

Here is the caller graph for this function:

6.1.1.23 write_to_file()

```
template<typename Container>
void write_to_file (
    const string & filename,
    const Container & studentai)
```

Definition at line 254 of file [duomenys.cpp](#).

Here is the caller graph for this function:

6.1.2 Variable Documentation

6.1.2.1 umap

```
std::unordered_map<int, string> umap
```

Definition at line 5 of file [duomenys.cpp](#).

6.2 duomenys.cpp

[Go to the documentation of this file.](#)

```
00001 #include "duomenys.h"
00002 #include "zmogus.h"
00003 #include "saugojimas_masyve.h"
00004
00005 std::unordered_map<int, string> umap;
00006
00007
00008
00009 // Getters
00010 string Studentai::getVardas() const { return vardas; }
00011 string Studentai::getPavarde() const { return pavarde; }
00012 vector<int> Studentai::getNd() const { return nd; }
00013 int Studentai::getEgz() const { return egz; }
00014 double Studentai::getVid() const { return vid; }
00015 double Studentai::getMed() const { return med; }
00016
00017 // Setters
00018 void Studentai::setVardas(const string &v) { vardas = v; }
00019 void Studentai::setPavarde(const string &p) { pavarde = p; }
00020 void Studentai::setNd(const vector<int> &n) { nd = n; }
00021 void Studentai::addNd(int n) { nd.push_back(n); }
00022 void Studentai::setEgz(int e) { egz = e; }
00023 void Studentai::setVid(double v) { vid = v; }
00024 void Studentai::setMed(double m) { med = m; }
00025 void Studentai::setReserveNd(int n) { nd.reserve(n); }
00026
00027 void Studentai::clear() {
00028     setVardas("");
00029     setPavarde("");
00030     egz = 0;
00031     vid = 0;
00032     med = 0;
00033     nd.clear();
00034 }
00035
00036 void Studentai::removeLastNd() {
00037     if (!nd.empty()) {
00038         nd.pop_back();
00039     }
00040 }
00041
00042 //Destruktor
00043 Studentai::~Studentai() {
00044     nd.clear();
00045     clear();
00046 }
00047
00048
00049 // Copy constructor
00050 Studentai::Studentai(const Studentai &other) :
00051     nd(other.nd),
00052     egz(other.egz),
00053     vid(other.vid),
00054     med(other.med) {
00055     setVardas(other.getVardas());
00056     setPavarde(other.getPavarde());
00057 }
00058
00059 // Copy assignment operator
00060 Studentai &Studentai::operator=(const Studentai &other) {
```

```

00061     if (this != &other) {
00062         setVardas(other.getVardas());
00063         setPavarde(other.getPavarde());
00064         nd = other.nd;
00065         egz = other.egz;
00066         vid = other.vid;
00067         med = other.med;
00068     }
00069     return *this;
00070 }
00071
00072 // Move constructor
00073 Studentai::Studentai(Studentai &&other) noexcept :
00074     nd(move(other.nd)),
00075     egz(other.egz),
00076     vid(other.vid),
00077     med(other.med) {
00078
00079     setVardas(move(other.getVardas()));
00080     setPavarde(move(other.getPavarde()));
00081     // Atstato reiksmes
00082     other.clear();
00083 }
00084
00085 // Move assignment operator
00086 Studentai &Studentai::operator=(Studentai &&other) noexcept {
00087     if (this != &other) {
00088         // Move strings from base class directly
00089         setVardas(move(other.getVardas()));
00090         setPavarde(move(other.getPavarde()));
00091         nd = move(other.nd);
00092         egz = other.egz;
00093         vid = other.vid;
00094         med = other.med;
00095
00096         //Atstato reiksmes
00097         other.clear();
00098     }
00099     return *this;
00100 }
00101
00102 // Output operator
00103 ostream &operator<<(ostream &os, const Studentai &student) {
00104     os << setw(20) << left << student.getVardas()
00105         << setw(20) << left << student.getPavarde()
00106         << setw(20) << left << fixed << setprecision(2) << student.getVid()
00107         << setw(20) << left << fixed << setprecision(2) << student.getMed();
00108     return os;
00109 }
00110
00111 // Input operator
00112 std::ifstream &operator>>(std::ifstream &in, Studentai &student) {
00113     if (!in) {
00114         return in;
00115     }
00116
00117     string line;
00118     try {
00119         getline(in, line);
00120     } catch (const std::exception &e) {
00121         cerr << "Error reading line: " << e.what() << endl;
00122     }
00123
00124     stringstream ss(line);
00125     string vardas, pavarde;
00126
00127     ss >> vardas >> pavarde;
00128     student.setVardas(vardas);
00129     student.setPavarde(pavarde);
00130
00131
00132     int paz;
00133     while (ss >> paz) {
00134         if (paz >= 0 && paz <= 10) {
00135             student.addNd(paz);
00136         }
00137     }
00138
00139     if (student.nd.size() == 0) {
00140         return in;
00141     }
00142
00143     student.setEgz(student.getNd().back());
00144     student.removeLastNd();
00145
00146     student.setVid(galutinis_vid(student.getNd(), student.getEgz()));
00147     student.setMed(galutinis_med(student.getNd(), student.getEgz()));

```

```

00148
00149     return in;
00150 }
00151
00152 // Equal operator
00153 bool operator==(const Studentai &a, const Studentai &b) {
00154     return a.getVardas() == b.getVardas() &&
00155         a.getPavarde() == b.getPavarde() &&
00156         a.getNd() == b.getNd() &&
00157         a.getEgz() == b.getEgz() &&
00158         a.getVid() == b.getVid() &&
00159         a.getMed() == b.getMed();
00160 }
00161
00162
00163
00164
00165
00166 template<typename Container>
00167 void readas(const string &filename, Container &studentai) {
00168     ifstream in(filename);
00169     try {
00170         if (!in) {
00171             throw runtime_error("Failed to open file: " + filename);
00172         }
00173     } catch (const std::exception &e) {
00174         cerr << e.what() << endl;
00175         terminate();
00176     }
00177     string line;
00178     getline(in, line);
00179     Studentai student;
00180     studentai.reserve(1000000);
00181     try {
00182         while (!in.eof()) {
00183             if (in > student) {
00184                 studentai.push_back(std::move(student));
00185             }
00186         }
00187     } catch (const std::exception &e) {
00188         cerr << "Error reading line: " << e.what() << endl;
00189     }
00190     in.close();
00191     cout << "read- baigta\n";
00192     cout << "Studentu skaicius: " << studentai.size() << endl;
00193 }
00194
00195 template<typename Container>
00196 double dalina(Container &studentai, Container &blogis) {
00197     auto start = chrono::high_resolution_clock::now();
00198     stable_sort(studentai.begin(), studentai.end(), [](const Studentai &a, const Studentai &b) {
00199         return a.getVid() < b.getVid();
00200     });
00201     auto end = chrono::high_resolution_clock::now();
00202     auto it = find_if(studentai.begin(), studentai.end(), [](const Studentai &student) {
00203         return student.getVid() >= 5;
00204     });
00205
00206     blogis.insert(blogis.end(), make_move_iterator(studentai.begin()), make_move_iterator(it));
00207     studentai.erase(studentai.begin(), it);
00208     cout << "dalina - baigta\n";
00209     return chrono::duration<double>(end - start).count();
00210 }
00211
00212 template<typename Container>
00213 void sortass(Container &studentai) {
00214     cout << "Pasirinkite pagal ka norite rusiuoti studentus: \n"
00215         << "1. Pagal vardą \n"
00216         << "2. Pagal pavardę \n"
00217         << "3. Pagal galutini bala (vidurkis) \n"
00218         << "4. Pagal galutini bala (mediana) \n";
00219     try {
00220         int pasirinkimas;
00221         cin >> pasirinkimas;
00222         if (cin.fail() || pasirinkimas < 1 || pasirinkimas > 4) {
00223             throw invalid_argument("Neteisingas ivestis.");
00224         }
00225
00226         switch (pasirinkimas) {
00227             case 1:
00228                 stable_sort(studentai.begin(), studentai.end(), [](const Studentai &a, const Studentai
&b) {
00229                     return a.getVardas() < b.getVardas();
00230                 });
00231                 break;
00232             case 2:
00233                 stable_sort(studentai.begin(), studentai.end(), [](const Studentai &a, const Studentai
&b) {
00234                     return a.getPavarde() < b.getPavarde();

```

```

00235         });
00236         break;
00237     case 3:
00238         stable_sort(studentai.begin(), studentai.end(), [](const Studentai &a, const Studentai
&b) {
00239             return a.getVid() > b.getVid();
00240         });
00241         break;
00242     case 4:
00243         stable_sort(studentai.begin(), studentai.end(), [](const Studentai &a, const Studentai
&b) {
00244             return a.getMed() > b.getMed();
00245         });
00246         break;
00247     }
00248 } catch (const exception &e) {
00249     cout << "Klaida: " << e.what() << endl;
00250 }
00251 }
00252
00253 template<typename Container>
00254 void write_to_file(const string &filename, const Container &studentai) {
00255     ofstream out(filename);
00256     out << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(20) << left <<
"Galutinis (Vid.) / Galutinis (Med.)" << endl;
00257     out << "-----" << endl;
00258     for (const auto &student: studentai) {
00259         out << student << "\n";
00260     }
00261     cout << "write - baigta\n";
00262     out.close();
00263 }
00264 }
00265
00266 template<typename Container>
00267 void split_into_two_containers(Container &studentai, Container &vargsiukai, Container &kietiakai) {
00268     for (auto it = studentai.begin(); it != studentai.end(); ++it) {
00269         if (it->getVid() < 5) {
00270             Studentai temp;
00271             temp = move(*it);
00272             vargsiukai.push_back(move(temp));
00273         } else {
00274             Studentai temp;
00275             temp = move(*it);
00276             kietiakai.push_back(move(temp));
00277         }
00278     }
00279     studentai.clear();
00280 }
00281
00282 // Testavimo funkcija
00283 void testavimas(vector<Studentai> &studentai) {
00284     cout << "Sukurti mokiniai testavimui.\n" << endl;
00285     make_file(studentai, test_file_location + "test.txt", 1, 5);
00286
00287     cout << "Nuskaityti mokinius is failo.(naudojamas input operatorius)\n" << endl;
00288     readas(test_file_location + "test.txt", studentai);
00289     cout << "\nInput operator veikia!\n";
00290
00291     if (studentai.empty()) {
00292         cout << "Nuskaitymas neveikia: (" << endl;
00293         return;
00294     }
00295
00296     // Test copy constructor
00297     cout << "\nCopy constructor:" << endl;
00298     Studentai copy_constructed(studentai[0]);
00299     cout << "Pradinis: " << studentai[0] << endl;
00300     cout << "Po copy: " << copy_constructed << endl;
00301     cout << "Copy constructor " << (copy_constructed == studentai[0] ? "veikia!" : "neveikia:") << endl;
00302
00303     // Test copy assignment
00304     cout << "\nCopy assignment operator:" << endl;
00305     Studentai copy_assigned;
00306     copy_assigned = studentai[0];
00307     cout << "Original: " << studentai[0] << endl;
00308     cout << "Po copy assigned: " << copy_assigned << endl;
00309     cout << "Copy assignment " << (copy_assigned == studentai[0] ? "veikia!" : "neveikia:") << endl;
00310
00311     // Test move constructor
00312     cout << "\nMove constructor:" << endl;
00313     Studentai to_move = studentai[0];
00314     Studentai move_constructed(std::move(to_move));
00315     cout << "Moved to: " << move_constructed << endl;
00316     cout << "Originali reiksme turi buti tuscia: "
00317         << (to_move.getVardas().empty() && to_move.getNd().empty() ? "tuscia!" : "ne tuscia:") <<
endl;
00318 }

```

```

00320
00321 // Test move assignment
00322 cout << "\nMove assignment operator." << endl;
00323 Studentai move_source = studentai[0];
00324 Studentai move_target;
00325 move_target = std::move(move_source);
00326 cout << "Moved to: " << move_target << endl;
00327 cout << "Originali reiksme turi buti tuscia: "
00328 << (move_source.getVardas().empty() && move_source.getNd().empty() ? "tuscia!" : "ne
tuscia:") << endl;
00329
00330 cout << "\nStudentai is failo: \n";
00331 write(studentai);
00332 cout << "\nOutput operator veikia!\n";
00333
00334 cout << "\nDestruktorius automatiskai bus iskviestas, kai visi objektai iseis is scope\n";
00335 }
00336
00337 void meniu(vector<Studentai> &studentai) {
00338     try {
00339         int a;
00340         cout << "norint iversti duomenis ranka, iveskite 1. \n"
00341              << "norint sugeneruoti pazymius, iveskite 2. \n"
00342              << "norint sugeneruotus pazymius ir vardus, iveskite 3. \n"
00343              << "norint nuskaityti vardus is failo, iveskite 4. \n"
00344              << "norint sugeneruoti arba atlikti tyrimus su duomenimis spauskite 5. \n"
00345              << "norint baigti darba, iveskite 6.";
00346         cin >> a;
00347         if (cin.fail() || a < 1 || a > 6) {
00348             throw invalid_argument("Neteisingas iverstis.");
00349         }
00350         string path;
00351         switch (a) {
00352             case 1:
00353                 ivedimas();
00354                 writel();
00355                 break;
00356             case 2:
00357                 generuoti_paz_ranka(studentai);
00358                 write(studentai);
00359                 break;
00360             case 3:
00361                 generuoti_vard(studentai, 2, 0);
00362                 write(studentai);
00363                 break;
00364             case 4: {
00365                 int pasirinkimas;
00366                 cout << "pasirinkite kuri faila norite nuskaityti: \n"
00367                      << "1. studentai10000.txt \n"
00368                      << "2. studentai100000.txt \n"
00369                      << "3. studentai1000000.txt \n";
00370                 cin >> pasirinkimas;
00371                 if (cin.fail() || pasirinkimas < 1 || pasirinkimas > 3) {
00372                     throw invalid_argument("Neteisingas iverstis.");
00373                 }
00374                 try {
00375                     switch (pasirinkimas) {
00376                         case 1:
00377                             read(test_file_location + "studentai10000.txt", studentai);
00378                             break;
00379                         case 2:
00380                             read(test_file_location + "studentai100000.txt", studentai);
00381                             break;
00382                         case 3:
00383                             read(test_file_location + "studentai1000000.txt", studentai);
00384                             break;
00385                         default:
00386                             cout << "Neteisingas pasirinkimas." << endl;
00387                             break;
00388                     }
00389                 } catch (const exception &e) {
00390                 }
00391                 if (studentai.empty()) {
00392                     cout << "Nepavyko nuskaityti failo." << endl;
00393                 } else {
00394                     sortas(studentai);
00395                     ss_write(test_file_location + "kursiokai.txt", studentai);
00396                 }
00397                 break;
00398             }
00399             case 5: {
00400                 umap[1] = test_file_location + "tyrimas_studentai1000";
00401                 umap[2] = test_file_location + "tyrimas_studentai10000";
00402                 umap[3] = test_file_location + "tyrimas_studentai100000";
00403                 umap[4] = test_file_location + "tyrimas_studentai1000000";
00404                 umap[5] = test_file_location + "tyrimas_studentai10000000";
00405                 int pasirinkimas1;

```

```

00406         cout << "1. sugeneruoti failus." << endl
00407             << "2. tirti failus" << endl
00408             << "3. testavimas" << endl;
00409     try {
00410         cin >> pasirinkimas1;
00411     } catch (exception &e) {
00412         cout << "Klaida: " << e.what() << endl;
00413     }
00414
00415     switch (pasirinkimas1) {
00416     case 1: {
00417         int paz_sk;
00418         cout << "Iveskite pazymiu skaiciu: ";
00419         cin >> paz_sk;
00420         if (cin.fail() || paz_sk < 2) {
00421             throw invalid_argument("Neteisingas ivestis.");
00422         }
00423         auto start = chrono::high_resolution_clock::now();
00424         make_file(studentai, test_file_location + "tyrimas_studentai1000.txt", 1000,
00425 paz_sk);
00426         make_file(studentai, test_file_location + "tyrimas_studentai10000.txt", 10000,
00427 paz_sk);
00428         make_file(studentai, test_file_location + "tyrimas_studentai100000.txt",
00429 100000, paz_sk);
00430         make_file(studentai, test_file_location + "tyrimas_studentai1000000.txt",
00431 1000000, paz_sk);
00432         make_file(studentai, test_file_location + "tyrimas_studentai10000000.txt",
00433 10000000, paz_sk);
00434         auto end = chrono::high_resolution_clock::now();
00435         chrono::duration<double> duration = end - start;
00436         cout << "laikas, kuri uztruko sugeneruoti visus duomenis: " << duration.count()
00437             << " seconds."
00438             << endl;
00439         break;
00440     }
00441     case 2: {
00442         double sorto_laikas = 0;
00443         cout << "Pasirinkite konteinerio tipa: \n"
00444             << "1. std::vector \n"
00445             << "2. std::list \n"
00446             << "3. std::deque \n";
00447         int pasirinkimas;
00448         cin >> pasirinkimas;
00449         if (cin.fail() || pasirinkimas < 1 || pasirinkimas > 3) {
00450             throw invalid_argument("Neteisinga ivestis.");
00451             break;
00452         }
00453         using ContainerType = vector<Studentai>;
00454         if (pasirinkimas == 1) using ContainerType = std::vector<Studentai>;
00455         else if (pasirinkimas == 2) using ContainerType = std::list<Studentai>;
00456         else if (pasirinkimas == 3) using ContainerType = std::deque<Studentai>;
00457         else cout << "Neteisingas pasirinkimas." << endl;
00458         ContainerType studentai;
00459         ContainerType blogis;
00460         ContainerType kietiakai;
00461
00462         // cout << "pasirinkite, kuria strategija norite naudoti: \n"
00463         //     << "1 Strategija \n"
00464         //     << "2 Strategija \n"
00465         //     << "3 Strategija \n";
00466         int strategija = 2;
00467         //cin >> strategija;
00468         if (cin.fail() || strategija < 1 || strategija > 3) {
00469             throw invalid_argument("Neteisinga ivestis.");
00470         }
00471
00472         cout << "Pasirinkite kuri faila norite nuskaityti: \n"
00473             << "1. studentai1000.txt \n"
00474             << "2. studentai10000.txt \n"
00475             << "3. studentai100000.txt \n"
00476             << "4. studentai1000000.txt \n"
00477             << "5. studentai10000000.txt \n";
00478         cin >> pasirinkimas1;
00479         if (cin.fail() || pasirinkimas1 < 1 || pasirinkimas1 > 5) {
00480             throw invalid_argument("Neteisingas ivestis.");
00481         }
00482         // if (pasirinkimas1 > 5 || pasirinkimas1 < 1 || !umap.count(pasirinkimas1) +
00483         //     ".txt") {
00484         //     throw invalid_argument("Failas " + umap[pasirinkimas1] + ".txt
00485         //         nerastas.");
00486         // }
00487         auto nuskaitymas_pradzia = chrono::high_resolution_clock::now();
00488         readas(umap[pasirinkimas1] + ".txt", studentai);
00489         auto nuskaitymas_pabaiga = chrono::high_resolution_clock::now();
00490         auto dalinimo_pradzia = chrono::high_resolution_clock::now();

```



```

00490         switch (strategija) {
00491             case 1:
00492                 split_into_two_containers(studentai, blogis, kietiakai);
00493                 break;
00494
00495             case 2: {
00496                 sorto_laikas = dalina(studentai, blogis);
00497                 break;
00498             }
00499             case 3: {
00500                 sorto_laikas = dalina(studentai, blogis);
00501                 break;
00502             }
00503             default: {
00504                 cout << "Neteisingas pasirinkimas." << endl;
00505                 break;
00506             }
00507         }
00508
00509
00510         auto dalinimo_pabaiga = chrono::high_resolution_clock::now();
00511
00512         cout << "Pazangus studentai: \n";
00513
00514         switch (strategija) {
00515             case 1: {
00516                 sortass(kietiakai);
00517                 break;
00518             }
00519             case 2: {
00520                 sortass(studentai);
00521                 break;
00522             }
00523             case 3: {
00524                 sortass(studentai);
00525                 break;
00526             }
00527             default: {
00528                 cout << "Neteisingas pasirinkimas." << endl;
00529                 break;
00530             }
00531         }
00532
00533         cout << "Nepazangus studentai: \n";
00534         sortass(blogis);
00535         switch (strategija) {
00536             case 1: {
00537                 write_to_file(umap[pasirinkimas1] + "_stud_rez.txt", kietiakai);
00538                 write_to_file(umap[pasirinkimas1] + "_blogi_rez.txt", blogis);
00539                 break;
00540             }
00541             case 2: {
00542                 write_to_file(umap[pasirinkimas1] + "_stud_rez.txt", studentai);
00543                 write_to_file(umap[pasirinkimas1] + "_blogi_rez.txt", blogis);
00544                 break;
00545             }
00546             case 3: {
00547                 write_to_file(umap[pasirinkimas1] + "_stud_rez.txt", studentai);
00548                 write_to_file(umap[pasirinkimas1] + "_blogi_rez.txt", blogis);
00549                 break;
00550             }
00551             default: {
00552                 cout << "Neteisingas pasirinkimas." << endl;
00553                 break;
00554             }
00555
00556             cout << "\n Nuskaitymo laikas: " << chrono::duration<double>(
00557                 nuskaitymas_pabaiga - nuskaitymas_pradzia).count() << "s" << endl;
00558             cout << "\n rusiavimo didejimo tvarka laikas: " << sorto_laikas << "s" << endl;
00559             cout << "\n dalinimo laikas: " << chrono::duration<double>(
00560                 dalinimo_pabaiga - dalinimo_pradzia).count() << "s" << endl;
00561             cout << "\n Is viso laiko: " << chrono::duration<double>(
00562                 nuskaitymas_pabaiga - nuskaitymas_pradzia).count() + sorto_laikas +
00563                 chrono::duration<
00564                 double>(
00565                     dalinimo_pabaiga - dalinimo_pradzia).count() << "s" << endl;
00566                 break;
00567             }
00568             case 3: {
00569                 testavimas(studentai);
00570                 break;
00571             }
00572             default: {
00573                 cout << "Neteisingas pasirinkimas." << endl;
00574                 break;
00575             }
00576         }
00577         case 6: {

```

```

00577         cout << "\nDarbas baigtas." << endl;
00578         break;
00579     }
00580     default:
00581         cout << "Neteisingas pasirinkimas." << endl;
00582         break;
00583     }
00584 } catch (const exception &e) {
00585     cout << "Klaida: " << e.what() << endl;
00586 }
00587 }
00588
00589 void galutinis_vid(vector<int> nd, int egz, vector<Studentai> &studentai) {
00590     // Calculate final grade using average
00591     Studentai student;
00592     double vid = 0;
00593     try {
00594         for (int i = 0; i < nd.size(); i++) {
00595             vid += nd[i];
00596         }
00597         vid = nd.empty() ? 0 : vid / nd.size();
00598
00599         student.setVid(0.4 * vid + 0.6 * egz);
00600         studentai.push_back(student);
00601     } catch (const exception &e) {
00602         cerr << "Error calculating average: " << e.what() << endl;
00603     }
00604 }
00605
00606 double galutinis_vid(vector<int> nd, int egz) {
00607     // skaiciuoja galutini bala pagal vidurki
00608     double vid = 0;
00609     try {
00610         for (int i = 0; i < nd.size(); i++) {
00611             vid += nd[i];
00612         }
00613         vid /= nd.size();
00614     } catch (const std::exception &e) {
00615         cerr << "Error calculating average: " << e.what() << endl;
00616     }
00617     return 0.4 * vid + 0.6 * egz;
00618 }
00619
00620 void galutinis_med(vector<int> nd, int egz, vector<Studentai> &studentai) {
00621     // Calculate final grade using median
00622     Studentai student;
00623     sort(nd.begin(), nd.end());
00624     double med;
00625     if (nd.size() == 0) {
00626         med = 0;
00627     } else if (nd.size() % 2 == 0) {
00628         med = (nd[nd.size() / 2 - 1] + nd[nd.size() / 2]) / 2.0;
00629     } else {
00630         med = nd[nd.size() / 2];
00631     }
00632     student.setMed(0.4 * med + 0.6 * egz);
00633     studentai.push_back(student);
00634 }
00635
00636 double galutinis_med(vector<int> nd, int egz) {
00637     // skaiciuoja galutini bala pagal mediana
00638     sort(nd.begin(), nd.end());
00639     double med;
00640     if (nd.size() == 0) {
00641         med = 0;
00642     } else if (nd.size() % 2 == 0) {
00643         med = (nd[nd.size() / 2 - 1] + nd[nd.size() / 2]) / 2.0;
00644     } else {
00645         med = nd[nd.size() / 2];
00646     }
00647     return 0.4 * med + 0.6 * egz;
00648 }
00649
00650 void read(const string &filename, vector<Studentai> &studentai) {
00651     ifstream in(filename); // atidarymas is failo
00652     string line;
00653     getline(in, line);
00654     while (getline(in, line)) {
00655         stringstream iss(line);
00656         Studentai student;
00657         string vardas, pavarde;
00658         iss >> vardas >> pavarde;
00659         student.setVardas(vardas);
00660         student.setPavarde(pavarde);
00661
00662         vector<int> nd;

```

```

00664         int paz;
00665         while (iss » paz) {
00666             if (paz >= 0 && paz <= 10) nd.push_back(paz);
00667         }
00668
00669         int egz = nd.back();
00670         nd.pop_back();
00671
00672         student.setNd(nd);
00673         student.setEgz(egz);
00674         student.setVid(galutinis_vid(nd, egz));
00675         student.setMed(galutinis_med(nd, egz));
00676
00677         studentai.push_back(move(student));
00678     }
00679     in.close(); //uzdaromas failas
00680 }
00681
00682 void ss_write(const string &filename, vector<Studentai> &studentai) {
00683     ofstream out(filename);
00684     if (!out) {
00685         throw runtime_error("Failed to open file: " + filename);
00686     }
00687     if (studentai.empty()) {
00688         cout << "No students to write to the file." << endl;
00689         return;
00690     }
00691     stringstream ss;
00692     ss << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(20) << left <<
00693         "Galutinis (Vid.) / Galutinis (Med.)" << endl;
00694     ss << "-----" << endl;
00695     for (const auto &student: studentai) {
00696         ss << student << endl;
00697     }
00698     out << ss.str();
00699     out.close();
00700 }
00701
00702 void write(vector<Studentai> &studentai) {
00703     cout << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(20) << left <<
00704         "Galutinis (Vid.) / Galutinis (Med.)" << endl;
00705     cout << "-----" << endl;
00706     for (const auto &student: studentai) {
00707         cout << student << endl;
00708     }
00709 }
00710
00711 void write_file(const string &filename, vector<Studentai> &studentai, int nd_skaicius) {
00712     ofstream out(filename);
00713     if (!out) {
00714         throw runtime_error("negalima atidaryti failo: " + filename);
00715     }
00716     stringstream ss;
00717     ss << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde";
00718     for (int i = 1; i <= nd_skaicius - 1; i++) {
00719         ss << setw(20) << left << "ND" + to_string(i);
00720     }
00721     ss << setw(20) << left << "EGZAMINAS" << endl;
00722     while (!studentai.empty()) {
00723         for (auto &i: studentai) {
00724             ss << setw(20) << left << i.getVardas()
00725                 << setw(20) << left << i.getPavarde();
00726             for (auto k: i.getNd()) {
00727                 ss << setw(20) << left << k;
00728             }
00729             ss << setw(20) << left << i.getEgz();
00730             ss << endl;
00731         }
00732         out << ss.str();
00733         studentai.erase(studentai.begin(), studentai.end());
00734     }
00735
00736     out.close();
00737 }
00738
00739 void make_file(vector<Studentai> &studentai, const string &filename, int mok_sk, int paz_sk) {
00740     generuoti_vard(studentai, paz_sk, mok_sk);
00741     write_file(filename, studentai, paz_sk);
00742 }
00743
00744 void generuoti_paz(Studentai &student, int paz_sk) {
00745     while (paz_sk < 2) paz_sk = rand() % 10 + 1;
00746     vector<int> nd;
00747     for (int i = 0; i < paz_sk; i++) {
00748         int paz = rand() % 10 + 1;
00749         nd.push_back(paz);
00750     }

```

```

00751     int egz = nd.back();
00752     nd.pop_back();
00753
00754     student.setNd(nd);
00755     student.setEgz(egz);
00756     student.setVid(galutinis_vid(nd, egz));
00757     student.setMed(galutinis_med(nd, egz));
00758 }
00759
00760 void generuoti_paz_ranka(vector<Studentai> &studentai) {
00761     Studentai student;
00762     while (true) {
00763         try {
00764             cout << "Iveskite studento vardą (iveskite 'p' norint užbaigti): ";
00765             string vardas;
00766             cin >> vardas;
00767             if (vardas == "p") break;
00768             student.setVardas(vardas);
00769
00770             cout << "Iveskite studento pavardę: ";
00771             string pavarde;
00772             cin >> pavarde;
00773             if (cin.fail()) {
00774                 throw invalid_argument("Neteisingas įvestis.");
00775             }
00776             student.setPavarde(pavarde);
00777
00778             int paz_sk = rand() % 10 + 1;
00779             generuoti_paz(student, paz_sk);
00780             studentai.push_back(move(student));
00781         } catch (const exception &e) {
00782             cout << "Klaida: " << e.what() << endl;
00783             cin.clear();
00784         }
00785     }
00786 }
00787
00788 void generuoti_vard(vector<Studentai> &studentai, int paz_sk1, int mok_sk) {
00789     int vard_sk;
00790     if (mok_sk != 0) vard_sk = mok_sk;
00791     else vard_sk = rand() % 100 + 1;
00792
00793     int paz_sk;
00794     if (paz_sk1 != 0) paz_sk = paz_sk1;
00795     else paz_sk = rand() % 10 + 1;
00796
00797     for (int i = 0; i < vard_sk; i++) {
00798         Studentai student;
00799         string vardas = string(1, char(rand() % 26 + 65)) + ".";
00800         string pavarde = string(1, char(rand() % 26 + 65)) + ".";
00801
00802         student.setVardas(vardas);
00803         student.setPavarde(pavarde);
00804
00805         vector<int> nd;
00806         for (int j = 0; j < paz_sk - 1; j++) {
00807             nd.push_back(rand() % 10 + 1);
00808         }
00809         int egz = rand() % 10 + 1;
00810
00811         student.setNd(nd);
00812         student.setEgz(egz);
00813         student.setVid(galutinis_vid(nd, egz));
00814         student.setMed(galutinis_med(nd, egz));
00815
00816         studentai.push_back(move(student));
00817     }
00818 }
00819
00820 void sortas(vector<Studentai> &studentai) {
00821     cout << "Pasirinkite pagal ką norite rusiuoti studentus: \n"
00822           << "1. Pagal vardą \n"
00823           << "2. Pagal pavardę \n"
00824           << "3. Pagal galutini bala (vidurkis) \n"
00825           << "4. Pagal galutini bala (mediana) \n";
00826     try {
00827         int pasirinkimas;
00828         cin >> pasirinkimas;
00829         if (cin.fail() || pasirinkimas < 1 || pasirinkimas > 4) {
00830             throw invalid_argument("Neteisingas įvestis.");
00831         }
00832
00833         switch (pasirinkimas) {
00834             case 1:
00835                 sort(studentai.begin(), studentai.end(), [](const Studentai &a, const Studentai &b) {
00836                     return a.getVardas() < b.getVardas();
00837                 });

```

```

00838         break;
00839     case 2:
00840         sort(studentai.begin(), studentai.end(), [](const Studentai &a, const Studentai &b) {
00841             return a.getPavarde() < b.getPavarde();
00842         });
00843         break;
00844     case 3:
00845         sort(studentai.begin(), studentai.end(), [](const Studentai &a, const Studentai &b) {
00846             return a.getVid() > b.getVid();
00847         });
00848         break;
00849     case 4:
00850         sort(studentai.begin(), studentai.end(), [](const Studentai &a, const Studentai &b) {
00851             return a.getMed() > b.getMed();
00852         });
00853         break;
00854     }
00855     } catch (const exception &e) {
00856         cout << "Klaida: " << e.what() << endl;
00857     }
00858 }

```

6.3 duomenys.h File Reference

```

#include <iostream>
#include <vector>
#include <iomanip>
#include <cstdlib>
#include <random>
#include <ctime>
#include <unordered_map>
#include <chrono>
#include <list>
#include <deque>
#include <fstream>
#include <wchar>
#include <sstream>
#include "zmogus.h"

```

Include dependency graph for duomenys.h: This graph shows which files directly or indirectly include this file:

Classes

- class [Studentai](#)

klases.

Functions

- void [menu](#) (vector< [Studentai](#) > &studentai)
- double [galutinis_vid](#) (vector< int > nd, int egz)
- double [galutinis_med](#) (vector< int > nd, int egz)
- void [read](#) (const string &filename, vector< [Studentai](#) > &studentai)
- void [ss_write](#) (const string &filename, vector< [Studentai](#) > &studentai)
- void [write](#) (vector< [Studentai](#) > &studentai)
- void [write_file](#) (const std::string &filename, std::vector< [Studentai](#) > &studentai, int nd_skaicius)
- void [make_file](#) (vector< [Studentai](#) > &studentai, const string &filename, int mok_sk, int paz_sk)
- void [generuoti_paz](#) ([Studentai](#) &student, int paz_sk)
- void [generuoti_paz_ranka](#) (vector< [Studentai](#) > &studentai)
- void [generuoti_vard](#) (vector< [Studentai](#) > &studentai, int paz_sk1, int mok_sk)
- void [sortas](#) (vector< [Studentai](#) > &studentai)

Variables

- const std::string [test_file_location](#) = TEST_FILE_LOCATION

6.3.1 Function Documentation

6.3.1.1 galutinis_med()

```
double galutinis_med (
    vector< int > nd,
    int egz)
```

Definition at line 636 of file [duomenys.cpp](#).

6.3.1.2 galutinis_vid()

```
double galutinis_vid (
    vector< int > nd,
    int egz)
```

Definition at line 606 of file [duomenys.cpp](#).

6.3.1.3 generuoti_paz()

```
void generuoti_paz (
    Studentai & student,
    int paz_sk)
```

Definition at line 744 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.3.1.4 generuoti_paz_ranka()

```
void generuoti_paz_ranka (
    vector< Studentai > & studentai)
```

Definition at line 760 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.3.1.5 generuoti_vard()

```
void generuoti_vard (
    vector< Studentai > & studentai,
    int paz_sk1,
    int mok_sk)
```

Definition at line 788 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.3.1.6 make_file()

```
void make_file (
    vector< Studentai > & studentai,
    const string & filename,
    int mok_sk,
    int paz_sk)
```

Definition at line 739 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.3.1.7 meniu()

```
void meniu (
    vector< Studentai > & studentai)
```

Definition at line 337 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.3.1.8 read()

```
void read (
    const string & filename,
    vector< Studentai > & studentai)
```

Definition at line 650 of file [duomenys.cpp](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.3.1.9 sortas()

```
void sortas (
    vector< Studentai > & studentai)
```

Definition at line 820 of file [duomenys.cpp](#).

Here is the caller graph for this function:

6.3.1.10 ss_write()

```
void ss_write (
    const string & filename,
    vector< Studentai > & studentai)
```

Definition at line 682 of file [duomenys.cpp](#).

Here is the caller graph for this function:

6.3.1.11 write()

```
void write (
    vector< Studentai > & studentai)
```

Definition at line 702 of file [duomenys.cpp](#).

Here is the caller graph for this function:

6.3.1.12 write_file()

```
void write_file (
    const std::string & filename,
    std::vector< Studentai > & studentai,
    int nd_skaicius)
```

6.3.2 Variable Documentation**6.3.2.1 test_file_location**

```
const std::string test_file_location = TEST_FILE_LOCATION
```

Definition at line 19 of file [duomenys.h](#).

6.4 duomenys.h

[Go to the documentation of this file.](#)

```
00001 #ifndef DUOMENYS_H
00002 #define DUOMENYS_H
00003
00004 #include <iostream>
00005 #include <vector>
00006 #include <iomanip>
00007 #include <cstdlib>
00008 #include <random>
00009 #include <ctime>
00010 #include <unordered_map>
00011 #include <chrono>
00012 #include <list>
00013 #include <deque>
00014 #include <fstream>
00015 #include <wchar>
00016 #include <sstream>
00017 #include "zmogus.h"
00018
```

```

00019 const std::string test_file_location = TEST_FILE_LOCATION;           // skirtas naudoti SU CMake
00020 // const std::string test_file_location = "";                         // skirtas naudoti BE CMake
00021
00022 using std::cout;
00023 using std::cin;
00024 using std::endl;
00025 using std::setw;
00026 using std::left;
00027 using std::fixed;
00028 using std::setprecision;
00029 using std::string;
00030 using std::vector;
00031 using std::fstream;
00032 using std::ifstream;
00033 using std::ofstream;
00034 using std::time;
00035 using std::srand;
00036 using std::rand;
00039
00040 class Studentai : public Zmogus {
00041 private:
00042     vector<int> nd;
00043     int egz;
00044     double vid;
00045     double med;
00046 public:
00047     // Default constructor
00048     Studentai() : egz(0), vid(0), med(0) {}
00050
00051     // Rule of Five
00052     ~Studentai() override;
00053     Studentai(const Studentai& other); // Copy constructor
00054     Studentai& operator=(const Studentai& other); // Copy assignment
00055     Studentai(Studentai&& other) noexcept; // Move constructor
00056     Studentai& operator=(Studentai&& other) noexcept; // Move assignment
00057
00058     // Operators
00059     friend std::ostream& operator<<(std::ostream& os, const Studentai& student);
00060     friend std::istream& operator>>(std::istream& in, Studentai& student);
00061     friend bool operator==(const Studentai &a, const Studentai &b);
00062
00063     //Base klases funkcijos
00064     string getVardas() const override;
00065     string getPavarde() const override;
00066     void setVardas(const string &v) override;
00067     void setPavarde(const string &p) override;
00068
00069     // Getters
00070     vector<int> getNd() const;
00071     int getEgz() const;
00072     double getVid() const;
00073     double getMed() const;
00074
00075     // Setters
00076     void setNd(const vector<int> &n);
00077     void addNd(int n);
00078     void setEgz(int e);
00079     void setVid(double v);
00080     void setMed(double m);
00081     void setReserveNd(int n);
00082
00083     void clear();
00084     void removeLastNd();
00085 };
00086
00087
00088
00089
00091
00092
00093 void meniu(vector<Studentai> &studentai);
00094
00095 double galutinis_vid(vector<int> nd, int egz);
00096
00097 double galutinis_med(vector<int> nd, int egz);
00098
00099 void read(const string &filename, vector<Studentai> &studentai);
00100
00101 void ss_write(const string &filename, vector<Studentai> &studentai);
00102
00103 void write(vector<Studentai> &studentai);
00104
00105 void write_file(const std::string &filename, std::vector<Studentai> &studentai, int nd_skaicius);
00106
00107 void make_file(vector<Studentai> &studentai, const string& filename, int mok_sk, int paz_sk);
00108

```



```

00109 void generuoti_paz(Studentai &student, int paz_sk);
00110
00111 void generuoti_paz_ranka(vector<Studentai> &studentai);
00112
00113 void generuoti_vard(vector<Studentai> &studentai, int paz_sk1, int mok_sk);
00114
00115 void sortas(vector<Studentai> &studentai);
00116
00117 #endif //DUOMENYS_H

```

6.5 pirma_uzd.cpp File Reference

```
#include "duomenys.h"
```

Include dependency graph for pirma_uzd.cpp:

Functions

- int [main](#) ()

Variables

- vector< [Studentai](#) > [studentai](#)

6.5.1 Function Documentation

6.5.1.1 main()

```
int main ()
```

Definition at line 6 of file [pirma_uzd.cpp](#).

Here is the call graph for this function:

6.5.2 Variable Documentation

6.5.2.1 studentai

```
vector<Studentai> studentai
```

Definition at line 4 of file [pirma_uzd.cpp](#).

6.6 pirma_uzd.cpp

[Go to the documentation of this file.](#)

```

00001 #include "duomenys.h"           // skirtas atsidasant per editoriu
00002 // #include "duomenys.cpp"       // skirtas atsidasant per terminala
00003
00004 vector<Studentai> studentai;
00005
00006 int main() {
00007     srand(time(nullptr));
00008     meniu(studentai);
00009     cout << "Iveskite bet ka, kad baigti darba.";
00010     cin.ignore(256, '\n');
00011     cin.get();
00012     return 0;
00013 }

```

6.7 README.md File Reference

6.8 saugojimas_masyve.cpp File Reference

```
#include "saugojimas_masyve.h"
```

Include dependency graph for saugojimas_masyve.cpp:

Functions

- int [main](#) ()

6.8.1 Function Documentation

6.8.1.1 main()

int main ()

Definition at line 5 of file [saugojimas_masyve.cpp](#).

Here is the call graph for this function:

6.9 saugojimas_masyve.cpp

[Go to the documentation of this file.](#)

```
00001 #include "saugojimas_masyve.h"
00002
00003 using namespace std;
00004
00005 int main () {
00006     ivedimas();
00007     writel();
00008     return 0;
00009 }
```

6.10 saugojimas_masyve.h File Reference

```
#include <string>
#include <iostream>
#include <iomanip>
#include <fstream>
#include <algorithm>
```

Include dependency graph for saugojimas_masyve.h: This graph shows which files directly or indirectly include this file:

Classes

- struct [duomenys_m](#)

Functions

- void [ivedimas](#) ()
- double [galutinis_vid1](#) (int j)
- double [galutinis_med1](#) (int j)
- void [write1](#) ()

Variables

- int [zmn_sk](#) = 0
- struct [duomenys_m](#) B [100]

6.10.1 Function Documentation

6.10.1.1 galutinis_med1()

```
double galutinis_med1 (
    int j)
```

Definition at line 65 of file [saugojimas_masyve.h](#).

Here is the caller graph for this function:

6.10.1.2 galutinis_vid1()

```
double galutinis_vid1 (
    int j)
```

Definition at line 56 of file [saugojimas_masyve.h](#).

Here is the caller graph for this function:

6.10.1.3 ivedimas()

```
void ivedimas ()
```

Definition at line 21 of file [saugojimas_masyve.h](#).

Here is the caller graph for this function:

6.10.1.4 write1()

```
void writel ()
```

Definition at line 78 of file [saugojimas_masyve.h](#).

Here is the call graph for this function: Here is the caller graph for this function:

6.10.2 Variable Documentation

6.10.2.1 B

```
struct duomenys_m B[100]
```

6.10.2.2 zmn_sk

```
int zmn_sk = 0
```

Definition at line 12 of file [saugojimas_masyve.h](#).

6.11 saugojimas_masyve.h

[Go to the documentation of this file.](#)

```
00001 #ifndef SAUGOJIMAS_MASYVE_H
00002 #define SAUGOJIMAS_MASYVE_H
00003
00004 #include <string>
00005 #include <iostream>
00006 #include <iomanip>
00007 #include <fstream>
00008 #include <algorithm>
00009
00010 using namespace std;
00011
00012 int zmn_sk = 0;
00013
00014 struct duomenys_m {
00015     std::string vardas, pavarde;
00016     int nd[100];
00017     int nd_sk;
00018     int egz;
00019 }B[100];
00020
00021 void ivedimas() {
00022     try {
00023         while (true) {
00024             duomenys_m student;
00025             cout << "iveskite mokinio varda (iveskite 'p' norint uzbaigti): ";
00026             cin >> student.vardas;
00027             if (student.vardas == "p") break;
00028             cout << "iveskite mokinio pavarde: ";
00029             cin >> student.pavarde;
00030             student.nd_sk = 0;
00031             cout << "iveskite pazymius uz namu darbus (iveskite -1 norint uzbaigti): ";
00032             while (true) {
00033                 int grade;
00034                 cin >> grade;
00035                 if (cin.fail()) {
00036                     throw runtime_error("Netinkamas pazymys");
00037                 }
00038                 if (grade == -1) break;
```

```

00039         if (grade >= 0 && grade <= 10) {
00040             student.nd[student.nd_sk++] = grade;
00041         }
00042     }
00043     cout << "iveskite egzmino pazymi: ";
00044     cin >> student.egz;
00045     if (cin.fail()) {
00046         throw runtime_error("Netinkamas egzmino pazymys");
00047     }
00048     B[zmn_sk++] = student;
00049 }
00050 } catch (const exception &e) {
00051     cerr << "Klaida: " << e.what() << endl;
00052     cin.clear();
00053 }
00054 }
00055
00056 double galutinis_vid1(int j) {
00057     double vid = 0;
00058     for (int i = 0; i < B[j].nd_sk; i++) {
00059         vid += B[j].nd[i];
00060     }
00061     vid /= B[j].nd_sk;
00062     return 0.4 * vid + 0.6 * B[j].egz;
00063 }
00064
00065 double galutinis_med1(int j) {
00066     std::sort(B[j].nd, B[j].nd + B[j].nd_sk);
00067     double med;
00068     if (B[j].nd_sk == 0) {
00069         med = 0;
00070     } else if (B[j].nd_sk % 2 == 0) {
00071         med = (B[j].nd[B[j].nd_sk / 2 - 1] + B[j].nd[B[j].nd_sk / 2]) / 2.0;
00072     } else {
00073         med = B[j].nd[B[j].nd_sk / 2];
00074     }
00075     return 0.4 * med + 0.6 * B[j].egz;
00076 }
00077
00078 void writel() {
00079     cout << setw(20) << left << "Vardas" << setw(20) << left << "Pavarde" << setw(20) << left << "Galutinis
(Vid.) / Galutinis (Med.)" << endl;
00080     cout << "-----" << endl;
00081     for (int i = 0; i < zmn_sk; i++) {
00082         cout << setw(20) << left << B[i].vardas << setw(20) << left << B[i].pavarde << setw(20) << left <<
fixed << setprecision(2) << galutinis_vid1(i) << setw(20) << left << " / " << setw(20) << left << fixed <<
setprecision(2) << galutinis_med1(i) << endl;
00083     }
00084 }
00085
00086
00087 #endif //SAUGOJIMAS_MASYVE_H

```

6.12 zmogus.h File Reference

#include <string>

Include dependency graph for zmogus.h: This graph shows which files directly or indirectly include this file:

Classes

- class [Zmogus](#)

6.13 zmogus.h

[Go to the documentation of this file.](#)

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003 #include <string>
00004
00005 using std::string;
00006
00007 class Zmogus {
00008 protected:
00009     string vardas;
00010     string pavarde;
00011
00012 public:
00013     virtual string getVardas() const = 0;

```

```
00014     virtual string getPavarde() const = 0;
00015     virtual void setVardas(const string &v) = 0;
00016     virtual void setPavarde(const string &p) = 0;
00017
00018     // Virtual destructor
00019     virtual ~Zmogus() = default;
00020 };
00021 #endif //ZMOGUS_H
```


Index

- ~Zmogus
 - Zmogus, [14](#)
- B
 - saugojimas_masyve.h, [37](#)
- dalina
 - duomenys.cpp, [18](#)
- duomenys.cpp, [17](#)
 - dalina, [18](#)
 - galutinis_med, [18](#)
 - galutinis_vid, [18](#)
 - generuoti_paz, [18](#)
 - generuoti_paz_ranka, [18](#)
 - generuoti_vard, [18](#)
 - make_file, [19](#)
 - menu, [19](#)
 - operator<<, [19](#)
 - operator>>, [19](#)
 - operator==, [19](#)
 - read, [19](#)
 - readas, [19](#)
 - sortas, [20](#)
 - sortass, [20](#)
 - split_into_two_containers, [20](#)
 - ss_write, [20](#)
 - testavimas, [20](#)
 - umap, [21](#)
 - write, [20](#)
 - write_file, [20](#)
 - write_to_file, [20](#)
- duomenys.h, [31](#)
 - galutinis_med, [32](#)
 - galutinis_vid, [32](#)
 - generuoti_paz, [32](#)
 - generuoti_paz_ranka, [32](#)
 - generuoti_vard, [32](#)
 - make_file, [32](#)
 - menu, [32](#)
 - read, [32](#)
 - sortas, [33](#)
 - ss_write, [33](#)
 - test_file_location, [33](#)
 - write, [33](#)
 - write_file, [33](#)
- duomenys_m, [13](#)
 - egz, [13](#)
 - nd, [13](#)
 - nd_sk, [13](#)
 - pavarde, [13](#)
 - vardas, [13](#)
- egz
 - duomenys_m, [13](#)
- galutinis_med
 - duomenys.cpp, [18](#)
 - duomenys.h, [32](#)
- galutinis_med1
 - saugojimas_masyve.h, [36](#)
- galutinis_vid
 - duomenys.cpp, [18](#)
 - duomenys.h, [32](#)
- galutinis_vid1
 - saugojimas_masyve.h, [36](#)
- generuoti_paz
 - duomenys.cpp, [18](#)
 - duomenys.h, [32](#)
- generuoti_paz_ranka
 - duomenys.cpp, [18](#)
 - duomenys.h, [32](#)
- generuoti_vard
 - duomenys.cpp, [18](#)
 - duomenys.h, [32](#)
- getPavarde
 - Zmogus, [14](#)
- getVaras
 - Zmogus, [14](#)
- ivedimas
 - saugojimas_masyve.h, [37](#)
- main
 - pirma_uzd.cpp, [35](#)
 - saugojimas_masyve.cpp, [36](#)
- make_file
 - duomenys.cpp, [19](#)
 - duomenys.h, [32](#)
- menu
 - duomenys.cpp, [19](#)
 - duomenys.h, [32](#)
- nd
 - duomenys_m, [13](#)
- nd_sk
 - duomenys_m, [13](#)
- operator<<
 - duomenys.cpp, [19](#)
- operator>>
 - duomenys.cpp, [19](#)

- operator==
 - duomenys.cpp, 19
- Paleidimo instrukcija, 1
- pavarde
 - duomenys_m, 13
 - Zmogus, 15
- pirma_uzd.cpp, 35
 - main, 35
 - studentai, 35
- read
 - duomenys.cpp, 19
 - duomenys.h, 32
- readas
 - duomenys.cpp, 19
- README.md, 35
- saugojimas_masyve.cpp, 35
 - main, 36
- saugojimas_masyve.h, 36
 - B, 37
 - galutinis_med1, 36
 - galutinis_vid1, 36
 - ivedimas, 37
 - write1, 37
 - zmn_sk, 37
- setPavarde
 - Zmogus, 14
- setVardas
 - Zmogus, 14
- sortas
 - duomenys.cpp, 20
 - duomenys.h, 33
- sortass
 - duomenys.cpp, 20
- split_into_two_containers
 - duomenys.cpp, 20
- ss_write
 - duomenys.cpp, 20
 - duomenys.h, 33
- Studentai, 14
- studentai
 - pirma_uzd.cpp, 35
- test_file_location
 - duomenys.h, 33
- testavimas
 - duomenys.cpp, 20
- umap
 - duomenys.cpp, 21
- vardas
 - duomenys_m, 13
 - Zmogus, 15
- write
 - duomenys.cpp, 20
 - duomenys.h, 33
- write1
 - saugojimas_masyve.h, 37
- write_file
 - duomenys.cpp, 20
 - duomenys.h, 33
- write_to_file
 - duomenys.cpp, 20
- zmn_sk
 - saugojimas_masyve.h, 37
- Zmogus, 14
 - ~Zmogus, 14
 - getPavarde, 14
 - getVardas, 14
 - pavarde, 15
 - setPavarde, 14
 - setVardas, 14
 - vardas, 15
- zmogus.h, 38